

Comparison of Boosted Perceptrons and Support Vector Machines

Joshua Frankie Rayo
2011-19612
CS 280 Programming Assignment 3
November 7, 2016

Contents

1	Introduction	1
2	Methodology	2
2.1	Creating the dummy dataset	2
2.2	The Splice and Banana Datasets	2
2.3	The Perceptron	2
2.4	The Boosted Perceptron	2
3	Results	3
3.1	Boosted Perceptron Population Accuracy	3
3.2	SVM Kernel and Parameter Settings	4
3.3	Comparison of SVM and Boosted Perceptron	5
4	Conclusion	5

1 Introduction

In this paper the performance of boosted perceptrons are compared against support vector machines. Performance measures of these classifiers include accuracy rate, training time and testing time. The former classifier are actually an ensemble of perceptrons that is trained using the adaptive boosting algorithm its individual performance. This is implemented using Python that makes computations in vector form and as easy as possible. A third-party library LibSVM is used to instantiate the support vector machines.

2 Methodology

The classifier is made using Python, a high level, general-purpose programming language that is relatively easy to learn and has very good features in list manipulation. Third-party libraries used are `numpy`, `matplotlib` and `libsvm`.

2.1 Creating the dummy dataset

The dummy dataset is easily formed using the function `numpy.random.multivariate_normal` that generates random points given the mean and covariance of a multivariate normal distribution. For the negative class, 100 points are randomly generated with mean $\mu_1 = (0, 0)$ and variance $\mathbf{I}_2$. For the positive class, 100 points are randomly generated with mean $\mu_2 = (10, 10)$ and variance $\mathbf{I}_2$. Each of the classes are divided into two for training and testing the classifiers.

The dummy dataset can be used primarily to test the classification accuracy of a single perceptron. Unknown vectors which are near μ_1 using the Euclidean distance should have a negative class and vectors near μ_2 should be assigned to the positive class.

2.2 The Splice and Banana Datasets

From the `banana` dataset, first 400 points are used in training and remaining 4900 points are used in testing. For the `splice` dataset, first 1000 training points and remaining 2175 test points are used.

2.3 The Perceptron

A Python class is created to simulate the perceptron classifier with a *maxitercnt* attribute that limits the iterations for the pocket (learning) algorithm. In training the classifier, function `classify(X, y)`, the bias is included in the weight vector $\mathbf{w}$. Consequently the input vector should also be extended into another dimension by adding 1 as its first element so that the $y = \mathbf{w} \cdot \mathbf{x}$ still holds. The function `predict(X, y)` returns accuracy of a perceptron instance given a test set, while the function `outputv(X)` assigns an unknown input to its class using the trained weight vector.

2.4 The Boosted Perceptron

Another Python class `Adaboost` is created to simulate the boosted perceptron with attribute K as the number of classifier instances (inducers). This class is composed of many perceptron instances in an object-oriented paradigm. Function `adabtrain(X, y)` trains the model with K inducers using the adaptive boosting algorithm. The function `adabpredict(X, y)` returns accuracy of the boosted perceptron given a test set, while the function `adaboutputv(X)` assigns an unknown input to its class using the weighted hypotheses of the inducers.

3 Results

3.1 Boosted Perceptron Population Accuracy

Initially, 1000 inducers are trained for this classifier. The plot below shows the classifier accuracy when first K inducers are used for computing the outputs.

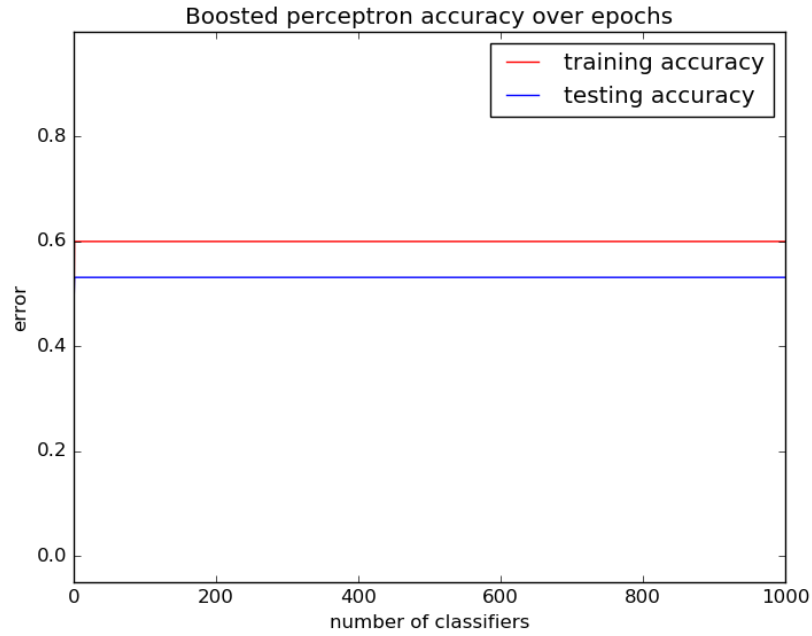


Figure 1: For the splice dataset, training and testing accuracy (60%, 52% respectively) remains constant given an arbitrary number of inducers. This is indeed a remarkable dataset for this classifier.

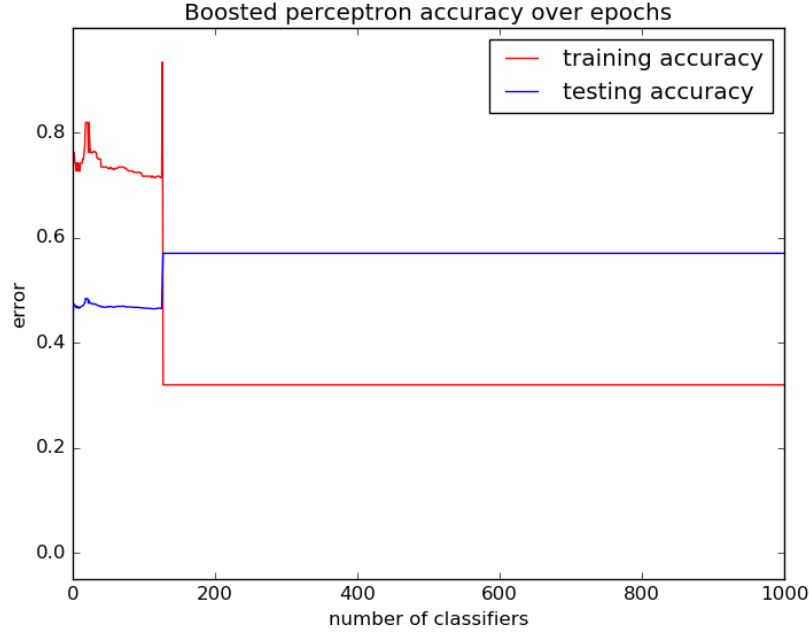


Figure 2: For the banana dataset, overfitting occurs when $K > 126$. It is recommended to use $K = 126$ since it makes training and testing accuracy high, 95% and 58% respectively.

3.2 SVM Kernel and Parameter Settings

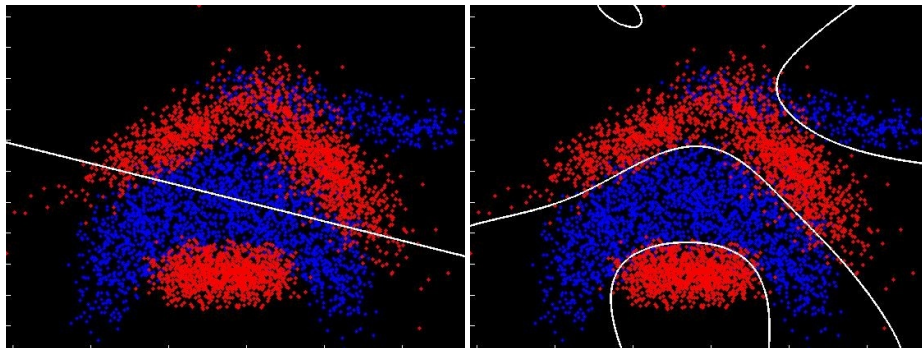
In training this classifier, one-class SVM (`-s 2`) is used since there are only two classes to be considered and different kernel function have been used, listed below:

- linear (`-t 0`)
- polynomial (`-t 1`)
- radial basis function (RBF) (`-t 2`)
- sigmoid (`-t 3`)

The table below lists the testing accuracy of SVM based on each kernel function with default coefficients and gamma values:

kernel/dataset	splice	banana
linear	56.2759	57.0612
polynomial	56.3218	57.0612
RBF	59.9080	57.2245
sigmoid	56.4138	57.0612

The RBF clearly bested the other kernel functions on both datasets. Moreover, the figures below show the linear and RBF decision curves for the banana dataset, found on Quora.



Clearly the decision curve from RBF kernel classifies the banana dataset better than the linear kernel. Its parameter is kept to its default value $\gamma = 1/\text{no_of_features}$.

3.3 Comparison of SVM and Boosted Perceptron

The accuracy rate and running time of classifiers are the performance measures taken into account. Using the best parameters for SVM and boosted perceptron, the table below shows the performance of each classifier over the `splice` and `banana` datasets:

classifier	dataset	testing accuracy	running time
boosted perceptron $K = 126$	splice	52	3hrs
	banana	58	1hr
SVM	splice	59.9080	0.8374 s
RBF kernel	banana	57.2245	0.2671 s

The results clearly show that support vector machines are still better than boosted perceptrons in terms of accuracy and running time.

4 Conclusion

Using the support vector machines generally take much less time in training and testing compared to using ensemble of classifiers that involves training large number of inducers. It is good to use SVM because of its inherent optimizing property. But if we can only use weak classifiers such as a perceptron, making an ensemble is a very good approach to make a better classifier than their individual performances.